



Lesson 1

Basic Elements of a Program



What is in this lesson?

Basic Programming Principles

Basic Program Elements

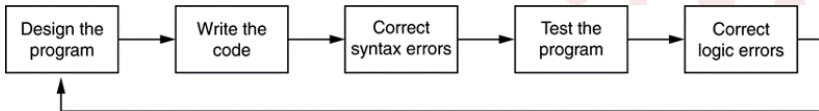
Basic Program Structure

Debugging and Testing

Basic Programming Principles

Designing a program

Program Development Cycle



Designing a program

- ▶ **Problem Analysis:** understand the task that the program is to perform
- ▶ **Solution Design:** determine the steps that must be taken to perform the task

What is an Algorithm?

- ▶ A set of well-defined logical steps that must be taken in order to perform a task.

What is an Algorithm?

- ▶ A set of well-defined logical steps that must be taken in order to perform a task.

Example

Write a program to calculate and display the gross pay for an hourly paid employee.

- ▶ Get the number of hours worked
- ▶ Get the hourly pay rate
- ▶ Multiply the number of hours worked by the hourly pay rate
- ▶ Display the result of the calculation performed in step 3

Pseudocode

Pseudocode is

- ▶ An informal language that has no syntax rules
- ▶ Not meant to be compiled or executed
- ▶ Used for designing programs, creating mock-ups of programs

Pseudocode

Pseudocode is

- ▶ An informal language that has no syntax rules
- ▶ Not meant to be compiled or executed
- ▶ Used for designing programs, creating mock-ups of programs

Write a program to calculate and display the gross pay for an hourly paid employee.

Input the hours worked

Input the hourly pay rate

Calculate gross pay as hours worked multiplied by pay rate

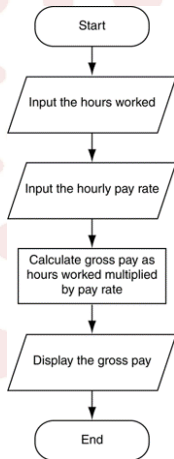
Display the gross pay

Flowcharts

A **Flowchart** is a tool that graphically depicts the steps that take place in a program.

Types of Symbols

- ▶ Ovals: terminal nodes
- ▶ Parallelograms: input and output nodes
- ▶ Rectangles: processing nodes
- ▶ Arrows: represent the flow



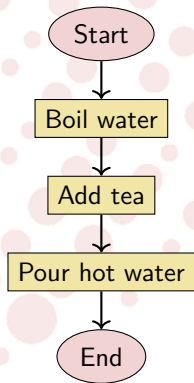
Sequence

Sequence is a control structure where instructions are executed **one at a time**, in the order they are written.

Sequence

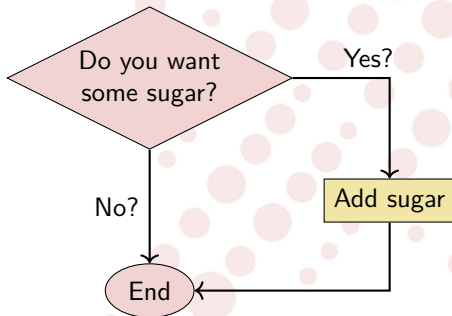
Sequence is a control structure where instructions are executed **one at a time**, in the order they are written.

Example: making a cup of tea



Selection

- ▶ A **selection** is a point where a decision is made.
- ▶ The program chooses **one path** to follow based on a **condition**.

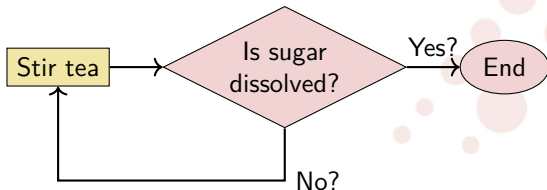


Iteration

Instead of writing the same instructions many times, we tell the computer to **loop**.

Definition

Iteration is a control structure that **repeatedly** executes a set of instructions **until a condition is satisfied**.



Modularisation

Modularisation is a method of designing programs by breaking down a problem into smaller, simpler, and independent components, called **modules**.

- ▶ Each module does **one specific task**
- ▶ Modules can be **combined** in a complete program
- ▶ Modules can be **reused** in other programs

Why is Modularisation important?

- ▶ Smaller problems are easier to solve
- ▶ Code is more reliable, easier to manage
- ▶ Code is easier to maintain

Basic Program Elements

Basic Data Types

- ▶ Strings: represent a 'string' of characters, e.g: "hello"
- ▶ Integers: integer numbers, e.g: 10
- ▶ Real (or floating point) numbers: real numbers, e.g: 3.142
- ▶ Booleans: carry one of two values True or False

Any of these values can be stored in the computer's memory using a **variable**.

We also have **constants**, whose values is set once and should never change after on. E.g: $\text{Pi} = 3.142$

Variables

A **variable** is a name that represents a value stored in the computer's memory.

We can create a variable with an assignment Statement:

Variable = Expression

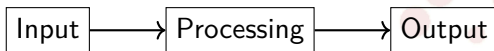
Variables naming rules

- ▶ Do not use Python key words (def, for, if,...)
- ▶ Do not contain spaces
- ▶ The first character be one of the capital letters or lowercase letters, or an underscore (_)
- ▶ From the second character, use lowercase letters, digits, underscore
- ▶ Uppercase and lowercase characters are distinct
- ▶ **The names must be meaningful that well represent the variables' use and the data they hold**

Operators and Statements

- ▶ **Operator:** A symbol or keyword that tells the computer to perform a specific operation on one or more values (operands).
- ▶ **Statement:** A single executable instruction that performs an action in a program. Programs are made up of sequences of statements.

Input and Output



- ▶ Python uses built-in functions to read input from the keyboard.
- ▶ A **function** is a piece of prewritten code that performs an operation and then returns a value back to the program.
- ▶ The **input** function can be used to read numeric data from the keyboard.
- ▶ The **print** function displays a string and then prints a newline character.

Basic Program Structure

Program Structure

A program is made up of:

- ▶ **Instructions:** tell the computer what action to take
- ▶ **Blocks:** group related instructions, usually responsible for same tasks
- ▶ **Comments:** explain the code

Instructions are written in an intentional order and executed from **top to bottom**.

Instructions must follow the programming language **syntax**.

- ▶ **Syntax** is a set of rules that must be strictly followed when writing a program.

Blocks

```
def main():  
    print("Hello")  
    print("World!")
```

```
main()
```

- ▶ A block groups related instructions.
- ▶ Blocks are usually grouped by task.
- ▶ Blocks make programs more readable.

Scope

- ▶ Variables, constants, and functions exist within a scope.
- ▶ A variable that has **global** scope is accessible to all blocks of code.
- ▶ A variable that has **local** scope is only accessible in the block in which, it was declared.

Program Starting Point

- ▶ The starting point determines where execution begins.
- ▶ Different programming languages define the starting point in different ways.
 - ▶ In **C**, program execution always begins at the function `main()`.
 - ▶ In **Python**, program execution begins at the first statement in global scope.

Comments

- ▶ Comments are ignored during compilation and execution.
- ▶ They explain the code in plain words to improve human readability.

Debugging and Testing

Debugging

Debugging is the process of finding and fixing errors (bugs) in a program.

- ▶ **Syntax errors:** break the programming language rules.
 - ▶ Misspelled word
 - ▶ Missing punctuation character
 - ▶ Incorrect use of an operator
- ▶ **Logic errors:** errors in reasoning or algorithm design that lead to wrong outputs.
 - ▶ Wrong formula
 - ▶ Wrong comparison ($>$ instead of $>=$)
 - ▶ Putting statements in the wrong order

Testing

Testing is the process of checking whether a program works as expected.

We run the program with known inputs and compare the results with expected outputs.

Testing ensures program correctness, reliability, and makes it easier to maintain and extend.

Recap

In this lecture, we've learned:

- ▶ Designing a program using pseudocode and flowcharts, concept of algorithm.
- ▶ Basic programming concepts: sequence, selection, iteration, and modularisation.
- ▶ Basic program elements and structures: data types, variables, operators, statements, blocks, scope.
- ▶ The importance of debugging and testing.

Further Reading

- ▶ T. Gaddis, Starting out with Python. Harlow: Pearson Education Limited, 2022.

Any Question?